UTDT

Tecnología Digital IV:
Redes de Computadoras

Trabajo Práctico
Servidor Web

Integrantes
Franco Arrieta
Gonzalo Benzaquen

Introducción

El objetivo de este trabajo práctico es poder demostrar los conocimientos adquiridos durante la cursada sobre el modelo TCP/IP, el funcionamiento de los sockets TCP, la transferencia de archivos utilizando el protocolo HTTP mediante un servidor web con la capacidad de tanto subir (modo upload) como descargar (modo download) imagenes, videos y archivos entre dispositivos conectados en una misma red.

En este contexto la capa de aplicación es la encargada de proporcionar tanto las interfaces como los protocolos necesarios para establecer una comunicación entre el programa y los clientes.

La finalidad de este servidor es escuchar las conexiones entrantes y atender las solicitudes HTTP de clientes que iniciaron esta conexión y permanecen conectados a la misma red para gestionar la transferencia de archivos entre el dispositivo conectado y el servidor. Para ello se genera una URL a la que se puede acceder con un código QR.

Implementación

Código QR

Al comenzar el desarrollo del trabajo, decidimos implementar la generación del QR para facilitar futuras pruebas desde nuestros celulares. Comenzamos investigando la librería *qrcode* para entender el comportamiento del mismo. Con esta investigación nos encontramos con funciones esenciales para llevar a cabo nuestra idea de funcionamiento del qr, entre ellas están `qr.add_data(...)` y `qr.make()`. Con esto en mente planteamos la idea en la función proporcionada `imprimir_qr_en_terminal(...)` función que le pasamos la url e imprime el código qr en ascii directamente en la terminal.

Uso de socket TCP

Para recibir y enviar información entre el servidor y el usuario primero necesitamos establecer una conexión. Para ello utilizamos sockets TCP encargados de escuchar, aceptar conexiones y manejar las peticiones HTTP que llegan desde el cliente. A la hora de generar este socket lo hicimos con un puerto mayor a 1023 para evitar usar puertos bien conocidos y lo bindeamos a la IP del dispositivo en el que se está corriendo. Así, el socket queda escuchando nuevas conexiones y cuando un cliente se conecte, se establecerá la conexión inicial que va a permitir intercambiar los mensajes HTTP necesarios para la carga o descarga de archivos

Para poder manejar varias conexiones en simultáneo, utilizamos el módulo *threading* de python. En nuestra implementación escuchamos constantemente en el thread principal (una especie de “listening socket”). Al llegar una conexión, creamos un thread nuevo encargado de resolver esa consulta mediante la función `resolver_conexion(...)`. Al finalizar la tarea, se cierra el thread paralelo.

Recepción de requests

Antes de pensar la lógica del servidor en algún modo específico, primero necesitamos mostrarle el html correspondiente al usuario. Para ello nos pareció importante encontrar alguna manera de manejar las peticiones HTTP de forma reutilizable, ya que sabíamos que lo necesitaríamos para todo el trabajo.

Para comenzar, leemos los headers de la request HTTP. Para ello al recibir una conexión TCP llamamos la función `leer_headers(...)`. Como sabemos que los headers HTTP terminan con *"\r\n\r\n"* podemos leer los bytes hasta encontrarnos con esta línea.

Esta implementación simple nos resultó útil al trabajar con las request de tipo GET iniciales, en las cuales el cuerpo de la petición es vacío. Sin embargo, al avanzar con el trabajo e introducir peticiones con un cuerpo HTTP (body) nos surgió un problema en el cuál se leía el header junto con el comienzo del cuerpo. Este problema fue difícil de detectar ya que surgió en secciones más avanzadas del trabajo en las cuales asumíamos que la función era correcta. La solución que encontramos fue extender la función `leer_headers()` para que también devuelva el comienzo del body e incluir este como el comienzo del cuerpo a leer.

Extracción de datos de los headers

Una vez obtenida la petición del cliente necesitamos entender qué tipo de consulta es para así devolver la respuesta apropiada.

En primer lugar debemos obtener el método HTTP que está entrando y la ruta solicitada. Para ello aprovechamos que estos datos estarán en la primera línea de los headers y los extraemos del string. De esta manera, podemos manejar las respuestas para cualquier tipo de request entrante, sea GET o POST, junto con la ruta. Una vez obtenidos estos datos, los usamos para generar la respuesta con la función `generar_respuesta_http(...)`.

Por otro lado, a la hora de manejar peticiones POST que incluían datos, necesitábamos leer bytes adicionales que estaban presentes en los headers, más específicamente el boundary y el content-length. Para ello utilizamos funciones de python, para splitear el string del header con los prefijos que nos permitían identificarlos (`"boundary="` y `"Content-Length: "`). Una vez obtenidos estos dos valores los utilizamos para leer el bytes enviados por el usuario en el cuerpo HTTP.

Generación de interfaz

Con la lectura de la ruta recibida y el método solicitado pudimos determinar las interfaces específicas a responder. Si recibimos un “GET /” simplemente generamos la interfaz html del modo en el que está corriendo el servidor (upload o download). En el caso de solicitar un GET de alguna ruta no definida, generamos una página de error 404 indicando al usuario que la página solicitada no ha sido encontrada.

Generación de headers de respuesta

Antes de enviar la respuesta al usuario, necesitamos incluir los headers HTTP correspondientes al tipo de contenido que estamos enviando. De esta manera, el receptor podrá interpretar el flujo de bytes entrante. Para esto, implementamos una función genérica `generar_headers_http(...)` la cuál recibe como parámetros el body de la respuesta y el estado HTTP de la respuesta (200 OK, 404 NOT FOUND, 403 FORBIDDEN, etc.). Con esta función incluimos como headers:

- *HTTP/1.1 + estado HTTP:* Para indicar que es una respuesta HTTP.
- *Content-Length: len(body):* Para que el receptor pueda interpretar la cantidad de bytes que deberá leer.

Luego, utilizamos el *Content-Type* para indicar que tipo de contenido estamos enviando.

En el caso de estar enviando código HTML: *text/html*

En el caso de estar enviando un archivo:

- *application/octet-stream*
- *Content-Disposition: attachment; filename="{archivo}"*: Para indicar el nombre del archivo enviado

Por otro lado, en el caso de estar utilizando compresión gzip también debemos incluir el siguiente header para indicarle al cliente que deberá descomprimir los bytes del cuerpo.

- *Content-Encoding: gzip*

Finalmente terminamos los headers con *"\r\n"* para indicar el final de los mismos y el comienzo del cuerpo. (También se incluye *"\r\n"* al final de cada línea del header, por lo que el header termina cuando se encuentran dos seguidos *"\r\n\r\n"*). Una vez generados los headers los codificamos en bytes y les agregamos el body, completando la respuesta HTTP.

Envío de respuesta por el socket

Una vez lista la respuesta HTTP utilizamos el socket para enviarlo. Luego de enviar los bytes, cerramos el socket (junto con el thread). Esta no era nuestra idea original. En un comienzo, teníamos en mente mantener abierto un solo socket por conexión. Al implementarlo, notamos varios problemas con esta arquitectura. Por ejemplo, luego de esperar unos segundos con el dispositivo conectado, este cerraba la conexión con el siguiente error: "*An existing connection was forcibly closed by the remote host*". Por otro lado, utilizando un único socket encontrábamos un error en el cuál el servidor detectaba muchas requests cuando el cliente había enviado sólo una. Por estas razones decidimos abrir y cerrar el socket por cada request, notando que este daba mejor funcionamiento cumpliendo con el objetivo del trabajo.

Funcionalidad de carga

Al recibir una petición de tipo POST en el modo upload, entendemos que el usuario nos ha enviado un archivo. Con esto en mente, utilizamos la función `manejar_carga()` para generar la respuesta del usuario y guardar el archivo en el servidor. Para obtener el contenido del archivo y su nombre, hacemos uso de la función proporcionada `parsear_multipart()` con el boundary obtenido de los headers. Luego simplemente la guardamos en el directorio asignado. Algo que notamos es que no era necesario chequear que el archivo ya exista en el directorio, ya que al escribir el archivo nuevo lo reemplazamos.

Una vez guardado el archivo, generamos una página de éxito para el usuario indicando que se ha subido exitosamente.

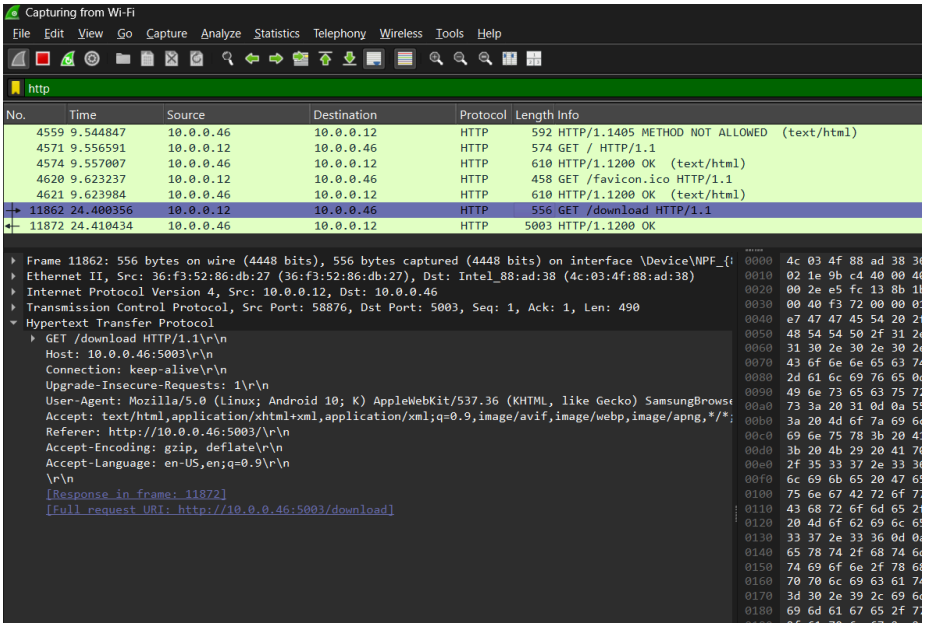
Funcionalidad de descarga

Al recibir una petición de tipo “GET /download” en el modo descarga, comenzamos el proceso de envío del archivo con la función `manejar_descarga()`. En el caso de que el archivo solicitado no exista en el directorio de archivos, se enviará una respuesta de error 404 con la interfaz correspondiente. En caso contrario, se leerán los bytes del archivo y serán enviados al cliente con los headers mencionados anteriormente. El usuario recibirá una notificación en su dispositivo indicando la descarga del archivo.

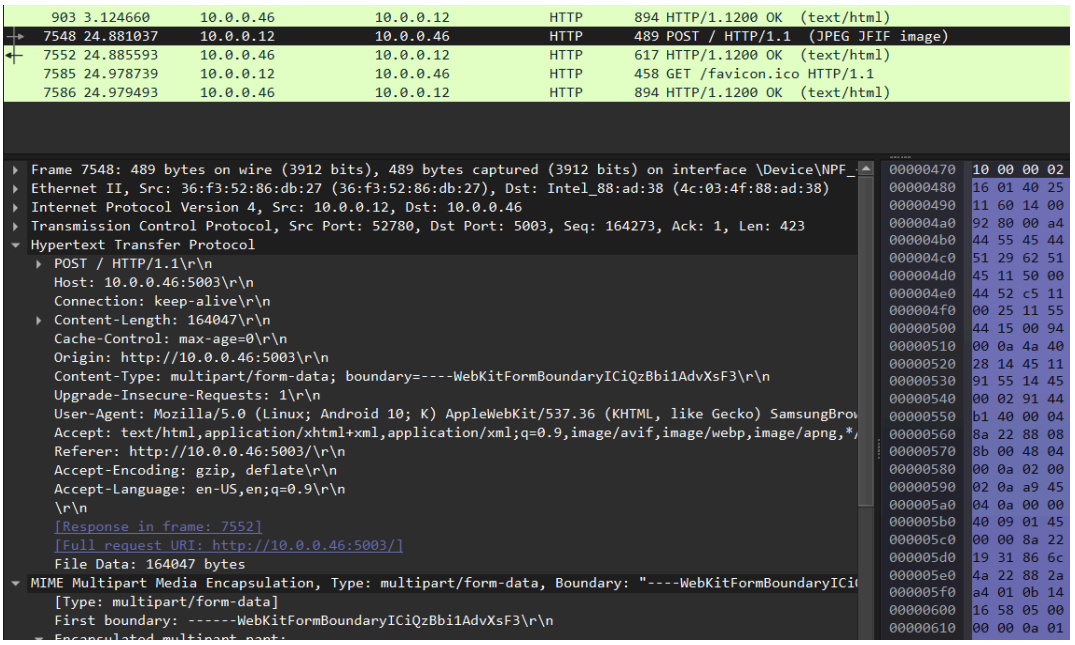
Request capturada con Wireshark

En las siguientes imágenes podemos observar las requests y las respuestas enviadas entre el cliente y el servidor.

En la primera imágen, podemos destacar los headers HTTP que envía el cliente al servidor (particularmente en la petición del GET /download).



En la segunda imágen capturamos el POST que envía el cliente con el archivo a subir. Podemos ver el contenido del archivo claramente separado por el boundary.



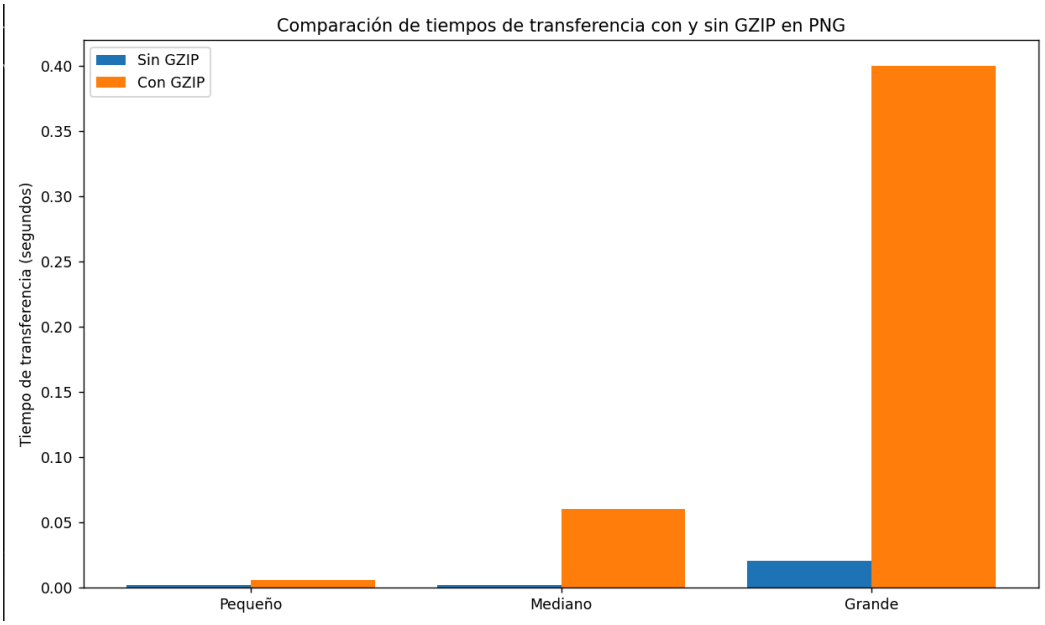
Compresión con gzip

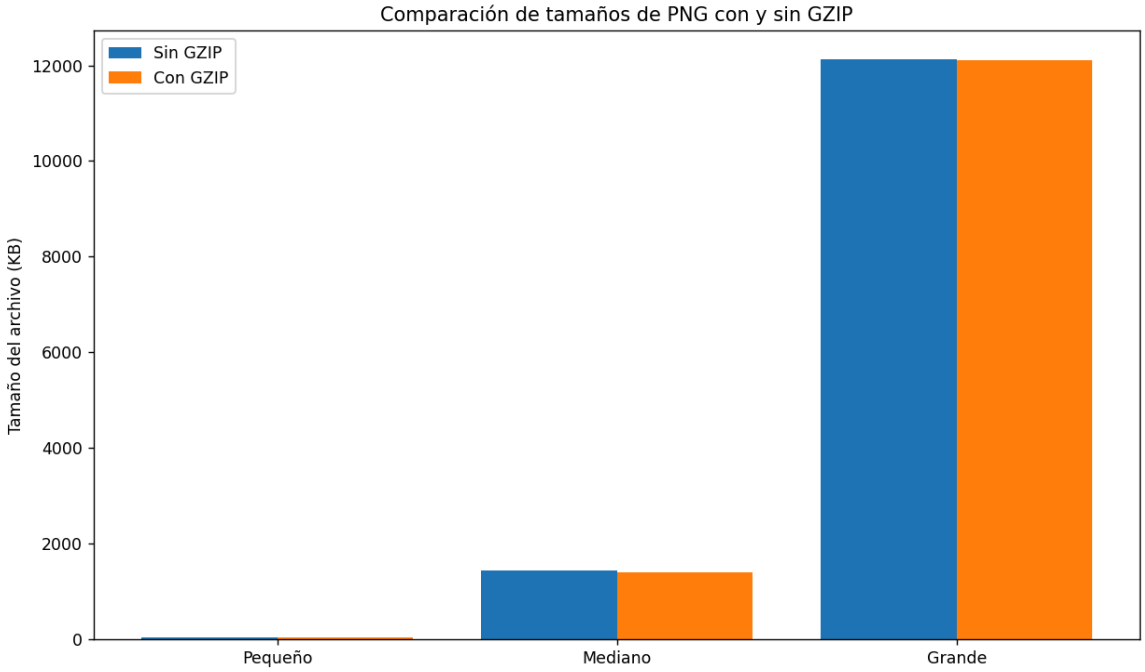
Para incluir compresión gzip a la hora de descargar archivos, debemos verificar que el usuario lo soporte. Esta información viene incluida en los headers de su request como `"Accept-Encoding: gzip"` en el caso de ser soportado (extraído de igual modo que los demás headers). Si el cliente lo acepta y el servidor está corriendo en este modo, se comprimen los bytes a ser enviados con la función `gzip.compress(body)` y se envían junto con los headers de gzip mencionados en el apartado **"Generación de headers de respuesta"**.

Experimentación con gzip

Para ver las diferencias de tiempos de transmisión y tamaños a la hora de descargar archivos usando o no gzip, decidimos comparar 3 tipos de archivos de diferentes tamaños. Los tipos de archivo que usamos son: PNG, PDF y TXT. Todas las pruebas fueron realizadas en la misma computadora corriendo el servidor (sin máquina virtual) y un mismo cliente, siendo este un celular conectado en la misma red local. Los resultados obtenidos están representados en los siguientes gráficos de barra y los ratios de compresión se encuentran en la tabla al final del apartado.

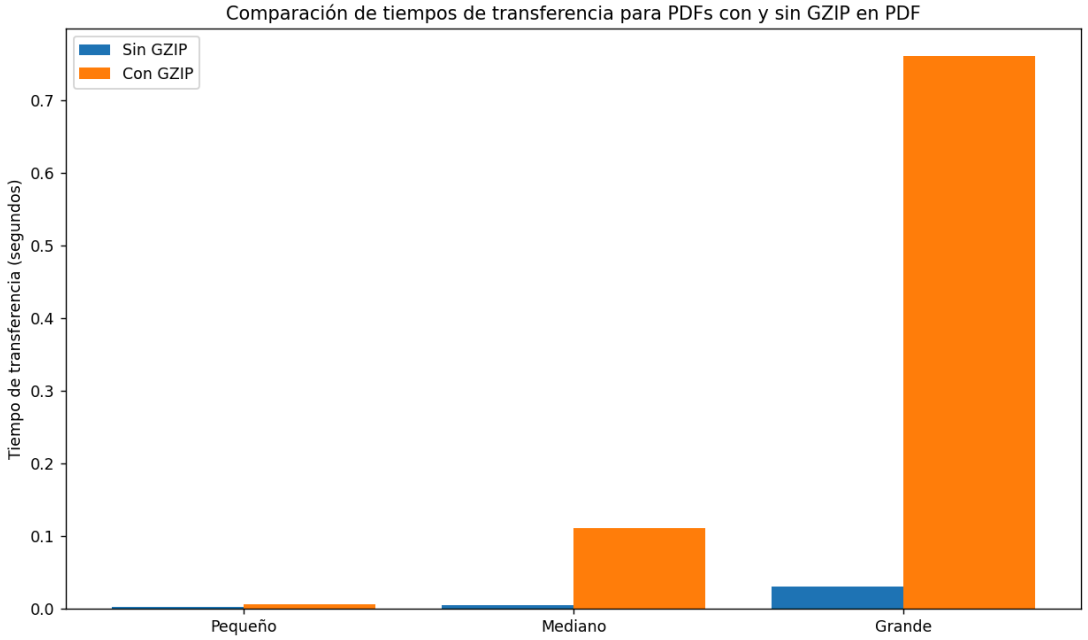
Archivos PNG

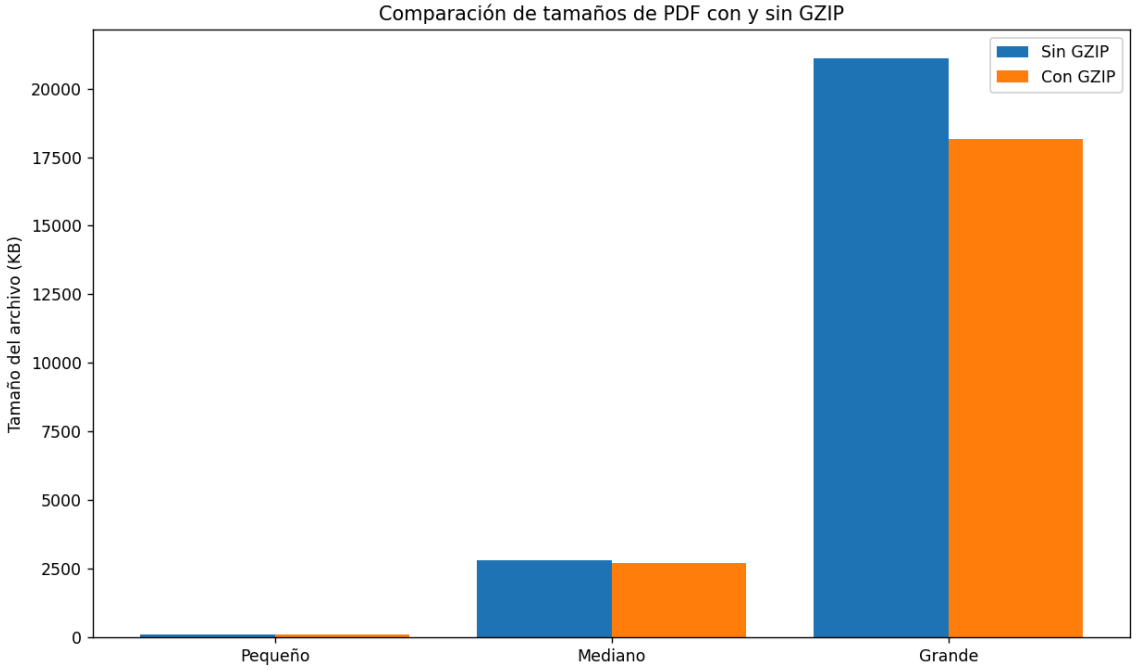




En el caso de los .png podemos ver que el tamaño de los archivos comprimidos es muy similar. Esto se debe a que los archivos .png ya presentan compresión. Por otro lado los tiempos de transferencia son mucho mayores cuando se tratan de archivos comprimidos ya que, aunque la cantidad de bytes transferidos es muy similar, se cuenta con el tiempo que tarda el archivo en ser comprimido.

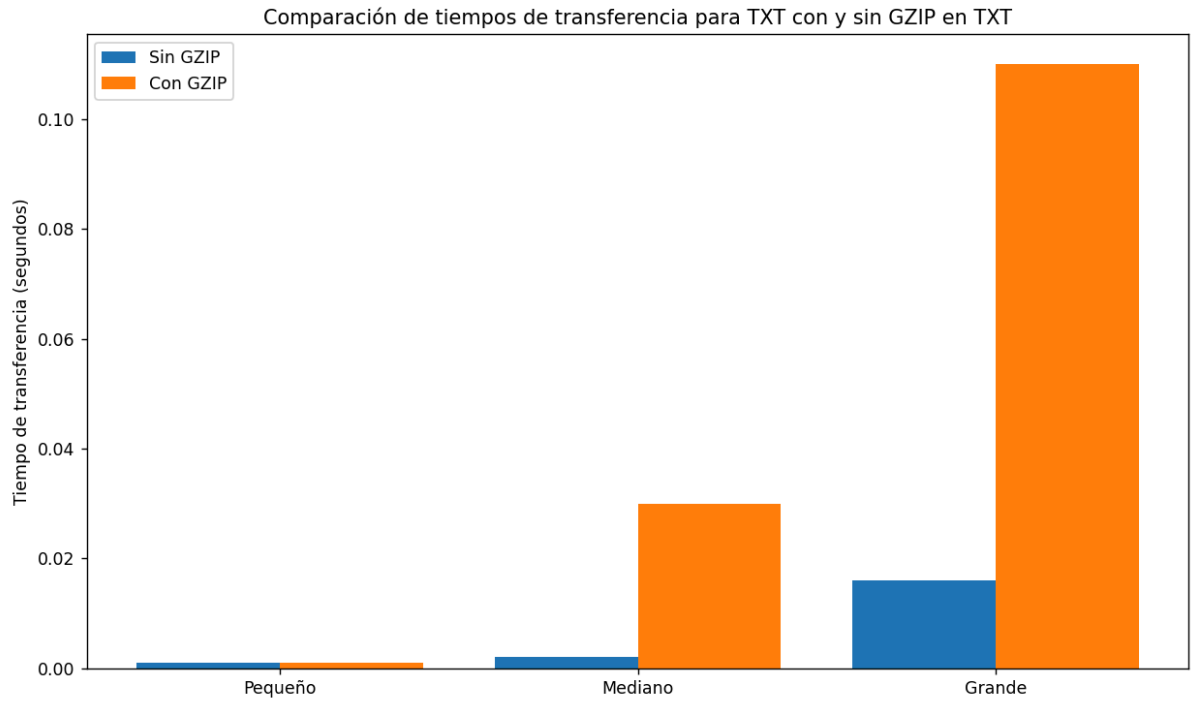
Archivos PDF

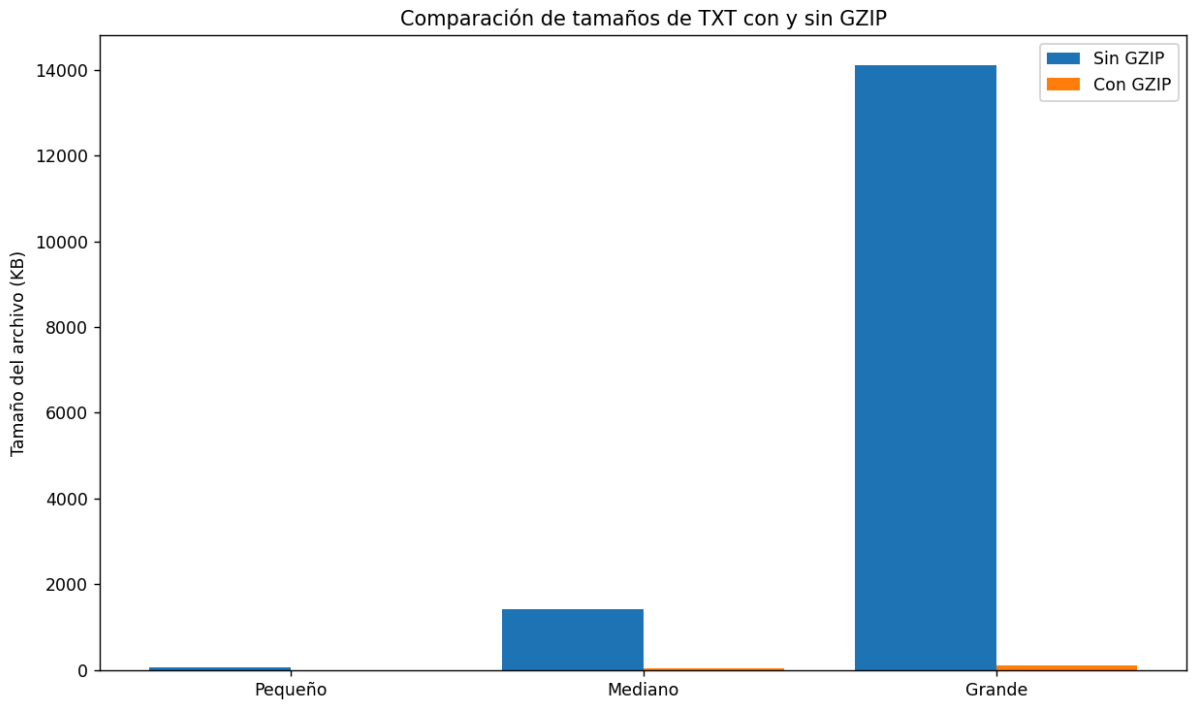




En el caso de los .pdf, podemos ver un comportamiento similar al de los .png. Sin embargo, con estos archivos sí notamos mayor compresión en los archivos más pesados. En cuanto a los tiempos de transferencia, replica la situación vista en los .png, donde los tiempos de transferencia son mayores a pesar de la compresión.

Archivos TXT





Finalmente, decidimos incluir compresión de archivos .txt en nuestra experimentación ya que notamos un comportamiento muy diferente a los casos anteriores donde trabajamos con archivos que ya habían sido comprimidos.

Como se puede ver en el gráfico, hay una gran diferencia entre el tamaño de los archivos. Esto tiene sentido ya que fueron generados copiando y pegando texto, por lo que presentan repetición de palabras. Esta característica permite un alto nivel de compresión con gzip.

Por otro lado, a pesar de reducir drásticamente el tamaño del archivo, los tiempos de transferencia siguen siendo mayores usando gzip. Creemos que esto se debe a la proximidad del celular con el servidor, como el delay de propagación es tan bajo sigue tardando menos evitar el tiempo de compresión y mandar el archivo completo.

Ratios de compresión observados

	Pequeño (<100 KB)	**Mediano (100 KB – 5 MB)**	**Grande (> 5 MB)**
PNG	-11.48%	-1.92%	-0.11%
PDF	-1.82%	-3.59%	-16.17%
TXT	-9117%	-4238%	-13975%

Análisis de seguridad

Con el funcionamiento actual de nuestro servidor, notamos muchas vulnerabilidades en la seguridad que podrían ser aprovechadas por actores maliciosos (Trudy's) con el fin de perjudicar tanto el intercambio de información como el funcionamiento del servidor. Particularmente podrían verse en riesgo la confidencialidad, integridad y disponibilidad del sistema.

En cuanto a la confidencialidad, un atacante podría sniffear nuestros paquetes con un programa como wireshark y leer la información que se transmite. Si quisiéramos enviar algún archivo confidencial, sería muy fácil para Trudy interceptar y copiarlo en su dispositivo. Como posible solución, se podría utilizar un sistema de clave simétrica entre el servidor y el cliente. De esta manera, enviaríamos los archivos encriptados logrando que si son interceptados sean ilegibles.

Por otro lado, desde el punto de vista de la integridad, se podría generar un ataque *man-in-the-middle* donde Trudy no solo lee la información enviada, sino que sería capaz de modificarla. Por ejemplo reemplazando el archivo enviado al servidor por un archivo malicioso. Para solucionarlo podríamos implementar un sistema de autenticación mediante encriptación con clave pública y privada (RSA por ejemplo). Si el servidor detecta que no está comunicándose con el cliente, podría automáticamente descartar los paquetes recibidos.

Por último, otro ataque que Trudy podría realizar es un *denial of service*. Podría generar un simple código que envía indefinidamente peticiones POST con archivos muy pesados a nuestro servidor. De esta manera, se generarían muchos procesos en paralelo hasta lograr

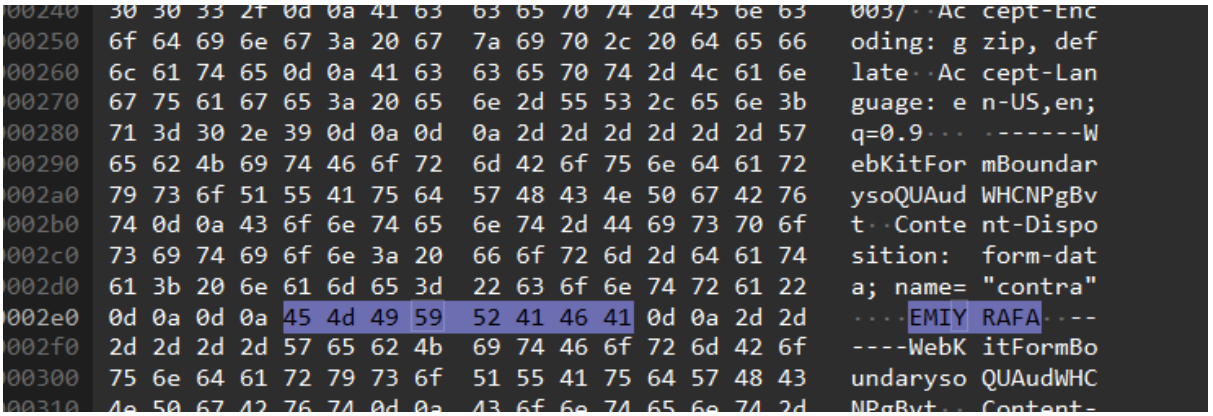
que el servidor se quede sin memoria y deje de funcionar. Para mitigarlo podríamos autenticar al cliente con un sistema de claves, o limitar la cantidad de solicitudes que el servidor puede recibir en simultáneo de un mismo host.

Una forma de mejorar la seguridad de nuestra conexión, sería utilizar el protocolo HTTPS en lugar de HTTP. De esta manera, haríamos uso del protocolo TLS en la comunicación contando con los beneficios que trae.

Implementación de seguridad

Como método básico de autenticación, decidimos implementar un sistema de contraseña. Esta contraseña deberá ser transmitida anteriormente al cliente (Por ejemplo, contándole personalmente a un amigo). De esta manera, el usuario deberá ingresar la contraseña **"EMIYRAFA"** antes de enviar el archivo al servidor. Si el servidor recibe una contraseña errónea (Por ejemplo, **"ANANKEYFACU"**) devolverá una página de error 403 indicando al usuario que el archivo ha sido rechazado.

De todas formas, cabe aclarar que este sistema es muy básico y fácilmente hackeable. Por ejemplo, en nuestras propias pruebas con wireshark, pudimos sniffear la contraseña fácilmente. La podemos ver capturada en la siguiente imágen:



Conclusión

En este trabajo se logró implementar un servidor web funcional operando tanto en la carga y descarga de archivos. A su vez, el TP nos permitió entender mejor el funcionamiento de la capa de aplicación, los sockets TCP y el intercambio de mensajes entre un servidor y cliente mediante el protocolo HTTP, también nos permitió entender la importancia de un buen manejo de errores en el servidor para asegurar el buen funcionamiento del mismo.

Este trabajo nos sirvió para poder conectar lo teórico con un funcionamiento real y para entender de una mejor forma como se comportan estas transferencias de archivos en una red.